Policies for Using Allocators in Library Classes

Document #:	P3002R1
Date:	2024-02-15 13:04 EST
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Pablo Halpern <phalpern@halpernwightsoftware.com>

Contents

§ 1 Abstract

To ensure consistency and cut down on excessive debate, LEWG is in the process of creating a standing document, SD-9, of design policies to be followed by default when proposing new facilities for the Standard Library; exceptions to these policies would require a paper author to provide rationale for each exception. Because the Standard

Library should give users control over the source of memory used by Library objects, policies on when and how classes should accept an allocator are badly needed so that allocator support does not become an afterthought or a topic of endless debate. This paper proposes a set of such policies.

§ 2 Change Log

R1: Reorganized and added additional motivation and information required to meet the policy-paper requirements specified in [P2267R1].

R0: Initial version.

§ 3 Motivation/Rationale

Memory management is a major part of building software. Numerous facilities in the C++ Standard library exist to give the programmer maximum control over how their program uses memory:

— `std::unique_ptr` and `std::shared_ptr` are parameterized with *deleter* objects that control, among other things, how memory resources are reclaimed.

— `std::vector` is preferred over other containers in many cases because its use of contiguous memory provides optimal cache locality and minimizes allocate/deallocate operations. Indeed, the LEWG has spent a lot of time on `flat_set` ([P1222R0]) and `flat_map` ([P0429R3]), whose underlying structure defaults to `vector` for this reason.

— Operators `new` and `delete` are replaceable, giving programmers global control over how memory is allocated.

— The C++ object model makes a clear distinction between an object's memory footprint and it's lifetime.

— Language constructs such as `void*` and `reinterpet_cast` provide fine-grained access to objects' underlying memory.

— Standard containers and strings are parameterized with allocators, providing object-level control over memory allocation and element construction.

This fine-grained control over memory that C++ gives the programmer is a large part of why C++ is applicable to so many domains — from embedded systems with limited memory budgets to games, high-frequency trading, and scientific simulations that require cache locality, thread affinity, and other memory-related performance optimizations.

An in-depth description of the value proposition for allocator-aware software can be found in [P2035R0]. Standard containers are the most ubiquitous examples of *allocator-aware* types. Their `allocator_type` and `get_allocator` members and allocator-parameterized

constructors allow them to be used like Lego® parts that can be combined and nested as necessary while retaining full programmer control over how the whole assembly allocates memory. For scoped allocators in particular (see Prior Art, below), having each element of a container support a predictable allocator-aware interface is crucial to giving the programmer the ability to allocate all memory from a single memory resource, such as an arena or pool. Note that the allocator is a *configuration* parameter of an object and does not contribute to its value.

In short, the principles underlying this policy proposal are:

1. **The Standard Library should be general and flexible**. To the extent possible, the user of a library class should have control over how memory is allocated.

2. **The Standard Library should be consistent**. The use of allocators should be consistent with the existing allocator-aware classes and class templates, especially those in the containers library.

3. **The parts of the Standard Library should work together**. If one part of the library gives the user control over memory allocation but another part does not, then the second part undermines the utility of the first.

4. **The Standard Library should encapsulate complexity**. Fully general application of allocators is potentially complex and is best left to the experts implementing the Standard Library. Users can choose their own subset of desirable allocator behavior only if the underlying Library classes allow them to choose their preferred approach, whether it be stateless allocators, statically typed allocators, polymorphic allocators, or no allocators.

5. **The Standard Library should try to provide simplifications of complex facilities**. The PMR part of the standard library provides a simplified allocator model for a common use case – that of providing an allocator to a class object and its subparts without infecting the object's type.

§ 4 Prior Art in the Standard

§ 4.1 Allocator Types

An allocator type is a class template having member functions `allocate` and `deallocate` and, optionally, `construct` and `destroy`. The requirements for allocator types are described in 16.4.4.6 [allocator.requirements][1].

Allocators can be divided into *stateless* and *stateful* categories, with stateful allocators subdivided into *shallow* and *scoped* subcategories. A shallow allocator controls affects only the top-level object to which it is passed whereas a scoped allocator affects all of its allocator-aware sub-objects as well. The Standard Library defines two allocators, `std::allocator`, which is stateless, and `std::pmr::polymorphic_allocator`, which is

scoped. It also defines `std::scoped_allocator_adaptor`, which makes shallow allocators into scoped allocators.

The `allocator_traits` class template provides a uniform interface for all allocator types, providing default operations and types for optional interface elements (see 20.2.9 [allocator.traits]).

§ 4.2 Polymorphic Allocators

Classic allocators are provided as template parameters, with behavior determined at compile time. This model does not always scale, as clients of an allocator-aware type also need to be templates to take advantage of the flexibility afforded by the allocator facility.

The `std::pmr::polymorphic_allocator` class template is a scoped allocator that acts as a bridge between the traditional statically selected allocator model and a runtime-selected allocator model. The mechanism by which a `polymorphic_allocator` manages memory is delegated to a *resource* object whose type is derived from `std::pmr::memory_resource`. Two container objects using `polymorphic_allocator` can have identical type yet allocate memory using different mechanisms. Moreover, a user-defined type can use allocators in its interface and implementation without being converted to a template, with all the complexity and scaling issues that entails. At the cost of virtual function dispatches for allocation and deallocation, polymorphic allocators provide a simpler model for many uses. See my CppCon 2017 talk, [Halpern2017], for more information about polymorphic allocators and how they simplify allocator-aware development.

§ 4.3 Allocator-aware Containers

Except for `array`, each container in the Standard Library supports an allocator template parameter and each of its constructors has a version that allows an allocator to be passed in at initialization. An allocator-aware container has an `allocator_type` member type and a `get_allocator` member function that returns the allocator. The `Allocator` parameter for all allocator-aware containers in the `std` namespace defaults to `std::allocator`.

Allocator-aware containers are required, per 24.2.2.5 [container.alloc.reqmts], to allocate memory and construct elements using the `allocate` and `construct` members of `allocator_traits`. Whether a container's allocator is copied during copy construction, copy assignment, move assignment, or swap is determined by the allocator's *propagation traits*, which are also accessed via `allocator_traits`.

Each allocator-aware container also has an alias in the `std::pmr` namespace for which the `allocator_type` is `std::pmr::polymorphic_allocator`:

```
namespace pmr {
  template <class T>
    class vector = std::vector<T, polymorphic_allocator<T>>;
}
```

The `pmr` alias makes the polymorphic allocators sub-model more accessible by defining a set of containers having the same scoped allocator type that can be nested arbitrarily, i.e,

```
std::pmr::monotonic_buffer_resource buff_rsrc;

// The list and all strings within it will allocate memory from buff_rsrc.
std::pmr::list<std::pmr::string> my_list(&buff_rsrc);
```

§ 4.4 Non-container Allocator Use

Allocators are used by `tuple`, `ranges::generator`, and `string_stream`, `sync_stream`, regular expressions, and `promise`. `tuple` is an interesting case because it does not allocate memory directly. Rather, `tuple`, might contain elements that use allocators and has constructors that allow an allocator to be forwarded to those elements. For example, `tuple<pmr::string, int, double>` can be constructed with an allocator that is passed to the first element. When an allocator is passed to a `tuple` constructor, each element is constructed by *uses-allocator construction* (20.2.8.2 [allocator.uses.construction]). Within the definition of *uses-allocator construction*, there is a special case for `std::pair` that allows an allocator to be passed to the pair's elements, even though `pair` is not otherwise allocator-enabled.

§ 5 Survey of Allocator Use in the Wider C++ Community

C++ allocators are used for a variety of purposes in various industries. Although statistics are hard to come by, below are some industry anecdotes.

— **Bloomberg (financial services)**: Bloomberg is probably the biggest proponent of allocator use in the community. Most of Bloomberg's C++ code base is allocator-enabled and allocators are used for a variety of purposes: cache locality, thread affinity, thread safety, shared memory, bug detection, and a type of garbage collection called *winking out*. Winking out is the act of abandoning an object without invoking its destructor and reclaiming all memory allocated by that object and its subobjects in a single operation. Though the informative paper [P2127R0][2] was originally a tutorial for Bloomberg engineers, the general concepts in its first few sections of can be useful for WG21 members interested in how allocator-aware software is put together.

— **Intel (enabling software for Intel microchips)**: Intel uses allocators to provide thread safety and thread affinity, as well as to allocate specific data structures in special-purpose memory pools such as high-bandwidth memory and accelerator memory.

— **Cradle (music production software)**: Cradle used allocators to provide deterministic execution time for allocating from a fixed buffer. They use `pmr::polymorphic_allocator` with `pmr::monotonic_buffer_resource` and

`std::pmr::unsynchronized_pool_resource` chained together to achieve determimism with ease of use.

— **Nvidia (GPUs)**: Nvidia uses allocators both for GPU memory and for local memory on CPUs. They use a custom approach similar to `pmr` that lets allocation be asynchronous with respect to a stream ("stream-ordered"). Mark Hoemmen said "The best part of this model is that users can control allocation behavior, without needing to think too much about it."

— **NI (computerized lab instrumentation)**: NI used an allocator in their kernel-mode drivers to provide page-locked memory, and another allocator that provides pageable memory. Their use was not entirely standards conforming.

— **Multiple companies (Cyclic Graph)**: Graphs with cycles create notoriously difficult memory-management problems. Using `shared_ptr` alone creates cycles that never get deallocated whereas using `weak_ptr` requires a separate registry to refer to all nodes. The entire graph can be managed, however, by allocating all of the nodes from a common pool- or arena- allocator, which reclaims all used memory at once when the graph is no longer needed. Note that this approach assumes that no resources besides memory are held by the nodes (because node destructors are not called). This approach implements a modest part of the design described by Herb Sutter in his CppCon 2016 keynote, [Sutter2016].

§ 6 Summary of Proposed Policies

The policies described in the Proposed SD-9 Wording section, below, are intended to ensure that new classes that allocate memory, directly or indirectly, are designed to enable the use of allocators to preform that allocation. It would be ideal if some or all of the *allocator-aware containers* requirements described in 24.2.2.5 [container.alloc.reqmts] were hoisted to their own section of the standard, describing all allocator-aware classes, not just containers. Such a change to the standard is outside the scope of this policy proposal, but is worth considering for the future, as it would make wording of these policies a bit simpler.

The most salient policy proposals are that:

— A class that might allocate per-object memory should be allocator aware.

— A class that contains members that might allocate per-object memory should accept an allocator on construction and forward it to those members' constructors.

An exception is made for situations where the overhead caused by an allocator parameter would be too great. Specifically, if the compile-time or runtime overhead of using `X<T, Alloc = std::allocator<>>` is excessive, then it might make sense to define non-allocator-aware class `X<T>` and a separate allocator-aware `basic_X<T, Alloc>` class. This approach is not encouraged, however, as it can lead to interoperability problems and

presents a confusing choice to the programmer. Note that, as with all policy exceptions, it is incumbent on the author of a library proposal to justify its application.

§ 7 Why Adopting these Policies would Improve Coherence and Save Time

§ 7.1 Advantages

— **Consistency and iteroperability**: Having a consistent allocator-aware infrastructure allows the various types that allocate memory to work together as building blocks that give the programmer control over how memory is allocated and used.

— **Time savings**: The LEWG has spent many hours debating, in person and on the reflector, whether this class or that should take an allocator. These policies should short-circuit most of those debates.

— **Better-initial-quality proposals**: There have been many instances in my memory where a type was proposed without allocators and needed to be reworked to enable allocator use. Having these policies in the *checklist* of LEWG policies should enable paper authors to come to the committee with higher-quality and more complete proposals.

§ 7.2 Disadvantages

— **Bigger interfaces**: Allocators make the class specification more complex.

— **More implementation complexity**: Allocators make implementation more difficult.

— **Longer compile times**: The flexibility afforded by the C++11 allocator model increases compile time by introducing metaprogramming based on `allocator_traits` and propagation traits.

In considering the first two disadvantages, bear in mind that our main task is to make C++ as useful as possible for programmers, not to make our job easier when writing interface specifications or implementing the Standard Library itself. It has been shown repeatedly that beginners can ignore allocators as a rule and tutorials for beginners can discuss allocators in more advanced sections.

The third issue can be mitigated significantly by specializing a class template specifically for `std::allocator`. Such a specialization need not use `allocator_traits` or consider propagation traits at all, and so should be almost as fast to compile (and run) as a non-allocator-aware class, yet the facility is still allocator-aware for users that want it.

§ 8 Proposed SD-9 Wording

Append to “List of Standard Library Policies” section of SD-9, the following policies. Note that the policy numbers are relative to the overall numbering of policies in SD-9.

1. A class that allocates memory should be *allocator-aware*, and should conform to as many of the *allocator-aware container* requirements (section 24.2.2.5 [container.alloc.reqmts]) as apply, even for non-contaianers — requirements on `T` are interpreted as requirements on any type potentially constructed within the allocated memory.

2. If it can be shown that applying the previous policy and making a class allocator-aware would incur excessive cost (in compile- and/or run-time), then the allocator-aware behavior can be separated out into a separate class, typically using the same name with the prefix, “`basic_`”.

3. The allocator type for an allocator-aware class should be optional and default to a specialization of `std::allocator`.

4. For each allocator-aware class template, there should be an alias in the `std::pmr` namespace where the allocator type is specified as `std::pmr::polymorphic_allocator`. If the allocator-aware class name begins with the `basic_` prefix, the `std::pmr` alias name should typically *not* have the prefix.

5. A class that contains subobjects (base classes or members) that might use allocators should accept an allocator, `A`, at construction amd forward it to those subobjects via *uses-allocator construction* with allocator `A` (20.2.8.2 [allocator.uses.construction]). If such a class, `X`, does not define `allocator_type`, then `std::uses_allocator<X, Alloc>` should be specialized to derive from `true_type` for all acceptable allocator types, `Alloc` (see 20.2.8.1 [allocator.uses.trait]). Such a class is called *allocator-enabled*, but is not necessarily *allocator-aware*.

6. An object within allocator-provided memory should be constructed by invoking `allocator_traits<Alloc>::construct`. For the purpose at this policy, a *dynamically sized* buffer is treated as allocated memory, even if it happens to reside in the object’s footprint when below a certain size (i.e., the small-object optimization); otherwise it is treated as a member variable (see previous policy).

7. If a class is allocator-aware or allocator-enabled, then every constructor should have a variant (via overloading or default arguments) that takes an allocator parameter. This quality enables *uses-allocator construction* in generic contexts (see 20.2.8.2 [allocator.uses.construction]).

8. An allocator-enabled *wrapper* class should, when possible, deduce an its `allocator_type` from the class it is wrapping.

§ 9 References

[Halpern2017] Pablo Halpern. Allocators, The Good Parts.
https://youtu.be/v3dz-AKOVL8?si=mi5JOJMaqD6lvRqv
[P0429R3] Zach Laine. 2016-08-31. A Standard flat_map.
https://wg21.link/p0429r3
[P1222R0] Zach Laine. 2018-10-02. A Standard flat_set.
https://wg21.link/p1222r0
[P2035R0] Pablo Halpern, John Lakos. 2020-01-13. Value Proposition: Allocator-Aware (AA) Software.
https://wg21.link/p2035r0
[P2127R0] Pablo Halpern. Making C++ Software Allocator Aware.
http://wg21.link/P2127R0
[P2267R1] Inbal Levi, Ben Craig, Fabio Fracassi. 2023-11-23. Library Evolution Policies.
https://wg21.link/p2267r1
[Sutter2016] Herb Sutter. Leak-Freedom in C++… By Default.
https://youtu.be/JfmTagWcqoE?si=OZ7EeziRKPaDnHSx

---

1. All citations to the Standard are to working draft N4971 unless otherwise specified.↩

2. Note that, due to some last-minute concerns, P2127 might not be published in time for the pre-Tokyo mailing, but should be available shortly after the mailing is sent out.↩